IFN647 Week 9 Review Questions
Solutions

Q1 (Multinomial Document Representation)
Table 1 shows an illustration of how documents are represented in the multinomial event space. In this table, there are 10 documents (each with a unique document id), two class labels (spam and not_spam), and a vocabulary that consists of the terms "cheap", "buy", "banking", "dinner", and "the".

Table 1. A training set (multinomial event space)

document *id*	cheap	buy	banking	dinner	the	*class*
1	0	0	0	0	2	not spam
2	3	0	1	0	1	spam
3	0	0	0	0	1	not spam
4	2	0	3	0	2	spam
5	5	2	0	0	1	spam
6	0	0	1	0	1	not spam
7	0	1	1	0	1	not spam
8	0	0	0	0	1	not spam
9	0	0	0	0	1	not spam
10	1	1	0	1	2	not spam

Based on Table 1, we can represent this training set for Naïve Bayes classification as follows:

$D = \{d_1, d_2, \ldots, d_{10}\}$, $\boldsymbol{C} = \{c_1, c_2\} = \{$'*spam*', '*not spam*'$\}$,

$V = \{w_1, w_2, \ldots, w_5\} = \{$'cheep', 'buy', 'banking', 'dinner', 'the'$\}$

where D is the training set, $\boldsymbol{C}$ is the set of class labels, and V is the set of terms (vocabulary).

We can use the following equation to calculate the condition probability $P(c|d)$

$$P(c|d) = \log\big(P(c)\big) + \sum_{w_i \in d} \log\left(P(w_i|c)\right)$$

(1) Based on Naïve Bayes Algorithm, calculate $P(c)$ for all $c \in \boldsymbol{C}$.
(2) Based on Naïve Bayes Algorithm, calculate $P(w_i|c_j)$ for all $w_i \in V$ and all $c_j \in \boldsymbol{C}$.

Solution:
(1)
$C_1 = \{d \in D, \text{label}(d) = c_1\} = \{d_2, d_4, d_5\}$
$C_2 = \{d \in D, \text{label}(d) = c_2\} = \{d_1, d_3, d_6, d_7, d_8, d_9, d_{10}\}$

$P(c_1) = Nc_1/N = |C_1| / |D| = 3/10 = 0.3$
$P(c_2) = Nc_2/N = |C_2| / |D| = 7/10 = 0.7$

(2)
By using Laplacian smoothed estimate:

$$P(w_i|c_j) = (tf_{wi,cj} + 1) /(|c_j| + |V|)$$

where $|c_j|$ is the total number of terms that occur in training documents with class label c_j and $tf_{wi,cj}$ is the number of times that term w_i occurs in class c_j in the training set.

$P(w_i\|c_j)$	cheep	buy	banking	dinner	the
c_1	(**10**+1)/(20+5)	(**2**+1)/25	(**4**+1)/25	(**0**+1)/25	(**4**+1)/25
c_2	(**1**+1)/(**15**+5)	(**2**+1)/20	(**2**+1)/20	(**1**+1)/20	(**9**+1)/20

=

$P(w_i\|c_j)$	cheep	buy	banking	dinner	the
c_1	11/25=0.44	3/25=0.12	5/25=0.2	1/25=0.04	5/25=0.2
c_2	2/20=0.1	3/20=0.15	3/20=0.15	2/20=0.1	10/20=0.5

Q2 (Naive Bayes classification)

Let $\{c_0, c_1\}$ be a set of classes and D be a training set, where each element in D is a pair (d_j, c_j), d_j is a document and c_j is its class. For example, Table 1 shows a training set D which contains 10 documents and their multinomial document representations.

Assume this training set is represented as two Python lists as follows:

D = [[0,0,0,0,2],[3,0,1,0,1],[0,0,0,0,1],[2,0,3,0,2],[5,2,0,0,1],[0,0,1,0,1],[0,1,1,0,1],[0,0,0,0,1],[0,0,0,0,1],[1,1,0,1,2]]

C = [0,1,0,1,1,0,0,0,0,0]

where a document is represented as a list of terms' frequency counts in the document (please note we assume the vocabulary set V= ['cheep', 'buy', 'banking', 'dinner', 'the'] is numbered from 0 to n-1), D is represented as a list of documents, C is represented as a list of class labels of the corresponding documents in D, and c_1 (spam) and c_0 (not spam) are labelled as 1 and 0 in C.

The training procedure of Naive Bayes classification is to calculates prior probability $P(c)$ and conditional probabilities $P(w|c)$ for all terms w in V and all classes c in $\{c_0, c_1\}$, where $P(c) = N_c / |D|$, $P(w|c) = (tf_{w,c} + 1)/(|c| + |V|)$, N_c is the number of training documents with class label c, $tf_{w,c}$ is the number of times that term w occurs in class c in the training set, and $|c|$ is the total number of times of terms that occur in training documents with class label c.

It then uses Bayes' Rule to compute the probability of class c_i occurring for the given document d, $P(c_i|d)$ (i =0 or 1) for all document d, and assigns a class label c_i to document d if c_i is the highest probability of being computed given the document.

(1) Define a python function *TRAIN_MULTINOMIAL_NB*(C, D, V) to calculate prior probability $P(c)$ and conditional probabilities $P(w_j|c_i)$. Please note the function parameters are different to the Naive Bayes Algorithm in the lecture notes.

(2) Define a python function *APPLY_MULTINOMIAL_NB*(*V, prior, condprob, d*) to assign a label (1 or 0) to document *d*, where *d* is represented as a list of words.

Solution:

(1)

```
# Define function to calculate the number of words in documents that labelled as class c in D

def n_words(c,C,D):
    n_w = 0
    for i in range(len(C)):
        if C[i]==c:
            for j in range(len(D[i])):
                n_w = n_w + D[i][j]
    return n_w

# The training algorithm, where we assume there are only two classes.
# It is different to the one defined in the lecture notes. It is good to understand the differences.
# For example, here we assume the document parsing is done and the set of terms V is provided.

def TRAIN_MULTINOMIAL_NB(C, D, V):
    # initialize variables
    N = len(D)
    prior = []
    condprob = tf = [[0 for w in range(len(V))] for c in range(2)]
    for c in range(2):
        Nc = 0
        for i in C:
            if C[i]==c:
                Nc = Nc+1
        prior.append(Nc/len(C))
        for w in range(len(V)):  # calculate tf(c,w)
            for d in range(len(D)):
                if C[d]==c:
                    tf[c][w] = tf[c][w] + D[d][w]
        for w in range(len(V)):
            condprob[c][w] = (tf[c][w]+1)/(n_words(c,C,D)+len(V))
    return(prior, condprob)
```

(2)

```
import math
def APPLY_MULTINOMIAL_NB(V, prior, condprob, d):
    W = {V[i]:i for i in range(len(V)) if V[i] in d}
    # W is a dict of term:number pairs
    print(W)
    score = {}
    for c in range(2):
        score[c]=math.log(prior[c])
        for (w,i) in W.items():
            score[c] = score[c] + math.log(condprob[c][i])
    if score[1]>score[0]:
        return 1
    else:
        return 0

# test the functions

# Assume the training set in Table 1 is represented as two Python lists as follows, where
```

```
# X_docs is the list of document vectors of documents in D, and
# y_class is the list of the corresponding classes of documents in D.
X_docs                                                                                                =
[[0,0,0,0,2],[3,0,1,0,1],[0,0,0,0,1],[2,0,3,0,2],[5,2,0,0,1],[0,0,1,0,1],[0,1,1,0,1],[0,0,0,0,1],
[0,0,0,0,1],[1,1,0,1,2]]
y_class = [0,1,0,1,1,0,0,0,0,0]
# We get 10 documents from D, and each document is a 5-dimentional vector
# The classes are c1 (spam) and c0 (not spam) and labelled as 1 and 0
V = ['cheep', 'buy', 'banking', 'dinner', 'the']
# test Q2 (1)
(prior, condprob) = TRAIN_MULTINOMIAL_NB(y_class, X_docs, V)
print(prior)
print(condprob)

# test Q2 (2)
d1 = ['cheep', 'buy', 'in', 'banking', 'ABC']
d2 = ['the', 'dinner', 'is', 'cheap']
y = APPLY_MULTINOMIAL_NB(V, prior, condprob, d1)
print(y)
```

Output

```
[0.7, 0.3]
[[0.1, 0.15, 0.15, 0.1, 0.5], [0.44, 0.12, 0.2, 0.04, 0.2]]
1
```

Q3 (***Rocchio* Classification**)

Let C be a set of classes and D be a training set, where each element in D is a pair (d_j, c_j), d_j is a document and c_j is its class. The training procedure of Rocchio classification for the input training set D can be found in the lecture notes. It uses centroids to define the boundaries between classes. The centroid of each class is computed as the vector average of its members.

For the given topic "R101", we have obtained the following training set which includes seven documents, ten selected features (terms) (VM, US, Spy, Sale, Man, GM, Espionag, Econom, Chief, and Bill) and the corresponding term weights, where Class = 1 means relevant, and Class = 0 means non-relevant.

Doc	VM	US	Spy	Sale	Man	GM	Espionag	Econom	Chief	Bill	***Class***
'39496'	0.17	0.01	0.20	0.00	0.02	0.20	0.12	0.11	0.00	0.00	1
'46547'	0.10	0.21	0.10	0.00	0.00	0.00	0.22	0.20	0.00	0.10	1
'46974'	0.00	0.23	0.10	0.20	0.05	0.10	0.10	0.10	0.01	0.00	1
'62325'	0.17	0.01	0.20	0.00	0.02	0.20	0.12	0.11	0.00	0.00	1
'6146'	0.10	0.00	0.00	0.30	0.10	0.20	0.00	0.12	0.10	0.00	0
'18586'	0.00	0.30	0.00	0.30	0.20	0.00	0.00	0.20	0.15	0.20	0
'22170'	0.20	0.00	0.00	0.15	0.20	0.25	0.00	0.00	0.00	0.10	0

Assume the above training set is represented as two Python dictionaries as follows:

document_vectors = {
'39496': {'VM': 0.17, 'US': 0.01, 'Spy': 0.20, 'Sale': 0.00, 'Man': 0.02, 'GM': 0.20, 'Espionag': 0.12, 'Econom': 0.11, 'Chief': 0.00, 'Bill': 0.00},

'46547': {'VM': 0.10, 'US': 0.21, 'Spy': 0.10, 'Sale': 0.00, 'Man': 0.00, 'GM': 0.00, 'Espionag': 0.22, 'Econom': 0.20, 'Chief': 0.00, 'Bill': 0.10},
'46974': {'VM': 0.00, 'US': 0.23, 'Spy': 0.10, 'Sale': 0.20, 'Man': 0.05, 'GM': 0.10, 'Espionag': 0.10, 'Econom': 0.10, 'Chief': 0.01, 'Bill': 0.00},
'62325': {'VM': 0.17, 'US': 0.01, 'Spy': 0.20, 'Sale': 0.00, 'Man': 0.02, 'GM': 0.20, 'Espionag': 0.12, 'Econom': 0.11, 'Chief': 0.00, 'Bill': 0.00},
'6146': {'VM': 0.10, 'US': 0.00, 'Spy': 0.00, 'Sale': 0.30, 'Man': 0.10, 'GM': 0.20, 'Espionag': 0.00, 'Econom': 0.12, 'Chief': 0.10, 'Bill': 0.00},
'18586': {'VM': 0.00, 'US': 0.30, 'Spy': 0.00, 'Sale': 0.30, 'Man': 0.20, 'GM': 0.00, 'Espionag': 0.00, 'Econom': 0.20, 'Chief': 0.15, 'Bill': 0.20},
'22170': {'VM': 0.20, 'US': 0.00, 'Spy': 0.00, 'Sale': 0.15, 'Man': 0.20, 'GM': 0.25, 'Espionag': 0.00, 'Econom': 0.00, 'Chief': 0.00, 'Bill': 0.10} }

relevance_judgements = {'R101': {'0': ['6146', '18586', '22170'], '1': ['39496', '46547', '46974', '62325']}}

Let *given_topic* = 'R101', design a Python function *train_Rocchio*(*document_vectors*, *relevance_judgements*, *given_topic*) to calculate the centroid of the relevant class (named as 'R101') and the centroid of non-relevant class (named as 'nR101'), respectively. For example, use the above table, we have 'nR101' = {'VM': 0.100, 'US': 0.100, 'Spy': 0.000, 'Sale': 0.250, 'Man': 0.167, 'GM': 0.150, 'Espionag': 0.000, 'Econom': 0.107, 'Chief': 0.083, 'Bill': 0.100}

Solution:

```
import sys, os

def is_belong_to(givendoc, topic_code, topic_docs):
    class_docs = topic_docs[topic_code][str(1)]
    return givendoc in class_docs

def train_Rocchio( train_set, topic_set, pos_cls):
        """Convert to dict of positive, negative docs.
        train_set: {docid:{term:tfidf}}
        topic_set: judgement for whole given collection
        pos_cls: the topic code, e.g. 'R101' """
        dvec_mean={}
        neg_dvec_mean = {}
        pos_n = 0
        neg_n = 0
        for (doc, dvec) in train_set.items():
            if is_belong_to(doc, pos_cls, topic_set):
                pos_n += 1
                for (term, score) in dvec.items():
                    try:
                        dvec_mean[term] += float(score)
                    except KeyError:
                        dvec_mean[term] = float(score)
            else:
                neg_n += 1
                for (term, score) in dvec.items():
                    try:
                        neg_dvec_mean[term] += float(score)
```

```
                except KeyError:
                    neg_dvec_mean[term] = float(score)  2 marks
        for t in list(neg_dvec_mean.keys()):
            neg_dvec_mean[t] /= float(neg_n)
        for t in list(dvec_mean.keys()):
            dvec_mean[t] /= float(pos_n)
        return { pos_cls: dvec_mean, ("!%s" % pos_cls)  : neg_dvec_mean }
```

```
# test in python
if __name__ == '__main__':

    #curr_idx ={'18586': {'quot': 0.053, 'passat': 0.144, 'saxoni': 0.222, 'new': 0.031, 'onli': 0.027,
'fix': 0.072, 'repurchas': 0.0727, 'agreement': 0.072}, '22513': {'iranian': 0.274, 'spi': 0.058,
'quot': 0.051}, '26642': {'peopl': 0.177, 'vessel': 0.146, 'capsiz': 0.170, 'nagav': 0.262, 'river':
0.170, 'southern': 0.127}, '26847': {'point': 0.122, 'index': 0.149, 'close': 0.132, 'stock': 0.141,
'trade': 0.067, 'share': 0.111}}
    #judge_set={'R101': {'0': ['6146', '18586', '22170', '22513', '26642', '26847', '27577', '30647',
'61329', '61780', '77909', '80425', '80950', '81463', '82912', '83167'], '1': ['39496', '46547',
'46974', '62325', '63261', '82330', '82454']}}

    document_vectors ={
        '39496': {'VM': 0.17, 'US': 0.01, 'Spy': 0.20, 'Sale': 0.00, 'Man': 0.02, 'GM': 0.20,
'Espionag': 0.12, 'Econom': 0.11, 'Chief': 0.00, 'Bill': 0.00},
        '46547': {'VM': 0.10, 'US': 0.21, 'Spy': 0.10, 'Sale': 0.00, 'Man': 0.00, 'GM': 0.00,
'Espionag': 0.22, 'Econom': 0.20, 'Chief': 0.00, 'Bill': 0.10},
        '46974': {'VM': 0.00, 'US': 0.23, 'Spy': 0.10, 'Sale': 0.20, 'Man': 0.05, 'GM': 0.10,
'Espionag': 0.10, 'Econom': 0.10, 'Chief': 0.01, 'Bill': 0.00},
        '62325': {'VM': 0.17, 'US': 0.01, 'Spy': 0.20, 'Sale': 0.00, 'Man': 0.02, 'GM': 0.20,
'Espionag': 0.12, 'Econom': 0.11, 'Chief': 0.00, 'Bill': 0.00},
        '6146': {'VM': 0.10, 'US': 0.00, 'Spy': 0.00, 'Sale': 0.30, 'Man': 0.10, 'GM': 0.20, 'Espionag':
0.00, 'Econom': 0.12, 'Chief': 0.10, 'Bill': 0.00},
        '18586': {'VM': 0.00, 'US': 0.30, 'Spy': 0.00, 'Sale': 0.30, 'Man': 0.20, 'GM': 0.00,
'Espionag': 0.00, 'Econom': 0.20, 'Chief': 0.15, 'Bill': 0.20},
        '22170': {'VM': 0.20, 'US': 0.00, 'Spy': 0.00, 'Sale': 0.15, 'Man': 0.20, 'GM': 0.25,
'Espionag': 0.00, 'Econom': 0.00, 'Chief': 0.00, 'Bill': 0.10} }

    relevance_judgements={'R101': {'0': ['6146', '18586', '22170'], '1': ['39496', '46547', '46974',
'62325']}}
    given_topic = 'R101'
    training_model = train_Rocchio(document_vectors, relevance_judgements, given_topic)
    print(training_model)
```

OUTPUT:

```
{'R101': {'VM': 0.11000000000000001, 'US': 0.115, 'Spy': 0.15000000000000002, 'Sale': 0.05, 'Man':
0.022500000000000003, 'GM': 0.125, 'Espionag': 0.13999999999999999, 'Econom': 0.13, 'Chief': 0.0025,
'Bill': 0.025}, '!R101': {'VM': 0.10000000000000002, 'US': 0.09999999999999999, 'Spy': 0.0, 'Sale':
0.25, 'Man': 0.16666666666666666, 'GM': 0.15, 'Espionag': 0.0, 'Econom': 0.10666666666666667, 'Chief':
0.08333333333333333, 'Bill': 0.10000000000000002}}
```

Q4 Which of the following is false for **sentiment analysis**? and justify your answer.

(1) Sentiment analysis (opinion mining) discovers users' opinions about products or services in on-line reviews or feedback or observes trends in public mood to analysis of clinical records.
(2) The orientation is the opinion provided about the entity and/or the aspect that was provided by the opinion holder at a specific time.
(3) The goal of emotion classification is to separate subjective from objective information, a binary classification task.
(4) Aspects are features, components or functions of the entity. They can be nouns and/or noun phrases
(5) Polarity classification is to group the expressed opinion in a document, a sentence or an entity feature/aspect in positive, negative or neutral regions.

Solution: (3)
The goal of emotion classification is to classify a piece of text according to a predefined set of basic emotions. The goal of subjectivity classification is to separate subjective from objective information, a binary classification task.

Q5 (Sentiment Analysis)
Consider the following sentence:
"The mobile phone has excellent build quality and performs efficiently when running games. Its camera quality is outstanding; however, the battery life is disappointing."

a) Using NLTK, define a get_aspect() function that identifies all *aspects* in the sentence by extracting tokens whose part-of-speech (POS) tags begin with **"NN"** (i.e., nouns and noun variants such as NN, NNS, NNP, NNPS). You may use the word_tokenize and pos_tag functions.
b) Design a prompt based on the Aspect-Based Sentiment Analysis (ABSA) model to extract all aspects mentioned in the sentence.
c) Compare and discuss the results obtained in a) and b), highlighting similarities, differences, and limitations of each approach.

Solution:

a)

```
from nltk import word_tokenize, pos_tag

def get_aspect(sentence):
    tokens = word_tokenize(sentence)
    tagged = pos_tag(tokens)
    aspects = [word for word, tag in tagged if tag.startswith("NN")]
    return aspects
```

```
sentence = "The mobile phone has excellent build quality and performs efficiently when running games. Its camera quality is outs
print(get_aspect(sentence))
```

```
['phone', 'quality', 'performs', 'games', 'camera', 'quality', 'battery', 'life']
```

b)

Prompt:

Please perform Unified Aspect-Based Sentiment Analysis task. Given the sentence, tag all aspects. Aspect should be substring of the sentence. If there are no aspects, return an empty list; otherwise return a python list of aspects (tokens) in single quotes. Please return python list only, without any other comments or texts.
Sentence: We discuss their opinions about the new stadium. Tokens: [‘stadium’]
Sentence: The new stadium is beautiful, and the design is inspired by the Queenslander architecture style. Tokens: [‘stadium’, ‘design’]
Sentence: The mobile phone has excellent build quality and performs efficiently when running games. Its camera quality is outstanding; however, the battery life is disappointing.
Tokens:

['mobile phone', 'build quality', 'camera quality', 'battery life']

c)

Both approaches can identify the relevant aspects in the sentence, indicating that method **a)** can capture what we are looking for. However, method **b)** produces more accurate and coherent results, particularly in terms of identifying complete and semantically meaningful aspect phrases.

Despite this improvement, an important limitation of method **b)** is its lack of transparency. While it outperforms method **a)** in effectiveness, it is not clear how the aspects are identified internally, as the underlying reasoning and decision process are not directly observable. In contrast, method **a)** follows an explicit and interpretable procedure based on part-of-speech tagging, but suffers from lower precision and limited semantic understanding. It appears feasible to incorporate additional rule-based constraints to extract meaningful bi-grams.

In summary, method **a)** offers greater interpretability but weaker performance, whereas method **b)** achieves better results at the cost of explainability.